Medico — Backend Modules Documentation

AI-Powered Early Health Detection Center — *backend/src* module reference

---

This document describes every C++ source file in `backend/src`, what each module is responsible for, the HTTP API routes it registers on the Crow web server, and notable implementation details (data structures, algorithms, and object-oriented design) used within it.

File	Module	Developer
main.cpp	Server entry point & route wiring	—
models.h	Shared data models & OOP hierarchy	—
auth_notification.h	Login, Sign Up & Send Notification	Yashwardhan Rohilla
ai_assessment.h	AI Integration (Gemini) & Start New Assessment	Vivek Chamoli
db_history.h	Database & Testing + Show Previous Assessment	Anjali
dashboard_triage.h	Dashboards + High Risk Individual triage	Nandini

1. main.cpp — Server Entry Point

Role: application bootstrap, configuration, route wiring, static file serving.

The single entry point for the whole backend. It does not implement any business logic itself — it wires together the four team modules and starts the Crow HTTP server.

What it does, in order:

- **Loads configuration** purely from environment variables (system env, falling back to parsing `backend/.env` by hand) via the `health::load_env_var` helper: MySQL host/port/user/password/database, the Gemini API key, and the HTTP listen port. Every value has a sane default if unset (e.g. port 8080, MySQL on 127.0.0.1:3306).
- **Constructs the four module objects**: `DatabaseManager`, `GeminiAIEngine`, and `AuthSessionManager`, then calls `db.init_database()` to connect to MySQL and auto-create the schema if needed.
- **Registers all REST routes** from the four modules onto one `crow::SimpleApp` instance: `register_db_history_routes`, `register_ai_assessment_routes`, `register_auth_notification_routes`, `register_dashboard_triage_routes`.
- **Serves the frontend** as static files: `read_frontend_file()` searches a handful of relative paths for files under `frontend/`, and dedicated routes serve `index.html`, `patient.html`, `worker.html`, plus any file under `/css/<file>` or `/js/<file>`.
- **Exposes a health-check endpoint** and finally starts the server with `app.port(port).multithreaded().run()`.

Routes registered directly in main.cpp:

Method & Path	Purpose
`GET /`	Serves the login/sign-up page (index.html)
`GET /index.html, /patient.html, /worker.html`	Serve the three frontend pages
`GET /css/<file>, /js/<file>`	Serves static CSS/JS assets
`GET /api/health`	Reports server status, MySQL connection state, and credits each team module

2. models.h — Shared Data Models

Role: plain data structures shared by every module, plus a small OOP demo hierarchy.

Core records:

- **User** — an account record (id, email/username, password hash, role `"patient"` or `"asha_worker"`, full name, phone, village/area, age, gender, created_at). `to_json()` serializes it for API responses.
- **Assessment** — one patient health-assessment record: clinical vitals (blood pressure, blood sugar, heart rate, oxygen level, temperature), free-text symptoms and medical history, the AI-produced risk score/level/diagnosis fields, and ASHA-worker follow-up state (alert sent, contact notes). Also has a `to_json()` serializer.
- **AIResult** — a smaller plain struct used as the return value of the Gemini AI call (risk score/level, possible chronic diseases, recommended specialist, precautions, immediate actions) before it gets merged into an `Assessment`.

OOP demo: Person / Patient / AshaWorker hierarchy

Layered on top of the plain `User` record purely to demonstrate inheritance and polymorphism (it is not used for storage or the API): an abstract `Person` base class exposes a virtual `role_description()`, overridden differently by `Patient` and `AshaWorker`. The helper `make_person(const User&)` builds the right subclass from a stored `User`'s role, and `auth_notification.h` calls it after every registration/login to print the role-specific description via dynamic dispatch through a `Person*` base pointer.

3. auth_notification.h — Login, Sign Up & Notification

Module 1 · Implemented by Yashwardhan Rohilla

Owns user authentication (registration, login, session validation) for both Patients and ASHA Workers, plus the ASHA worker's "send emergency alert" action.

AuthSessionManager

An in-memory, mutex-protected session store (`std::unordered_map<token, User>`). `create_session()` generates a random 32-character `sess_...` bearer token on successful login/registration; `get_user()` resolves a token back to a `User` for authenticated requests; `destroy_session()` handles logout. Sessions live only for the process lifetime (no persistence/expiry logic).

Routes registered:

Method & Path	Purpose
`POST /api/auth/register`	Sign up: validates email/password/full name, rejects duplicate accounts, creates the user v
`POST /api/auth/login`	Authenticates email + password against DatabaseManager and opens a new session
`GET /api/auth/me`	Resolves the Authorization: Bearer <token> header to the
`POST /api/auth/logout`	Destroys the caller's session token
`POST /api/worker/assessments/<id>/alert`	ASHA worker action: dispatches a high-risk alert (default or custom message) for a given a

4. ai_assessment.h — AI Integration & New Assessment

Module 2 · Implemented by Vivek Chamoli

Integrates Google Gemini as the sole diagnostic engine for turning a patient's reported symptoms and history into a structured clinical risk assessment. There is intentionally no offline/rule-based fallback — if Gemini is unavailable, the endpoint reports failure rather than guessing.

load_env_var()

A small free function that resolves a key first from the system environment (`std::getenv`), then by hand-parsing a `.env`-style file found at one of several candidate relative paths. This is the function `main.cpp` now reuses for all of its own configuration (MySQL credentials, server port, Gemini key).

GeminiAIEngine

- Holds the API key (constructor argument, or looked up via `load_env_var` if none given) and performs HTTPS calls to the Gemini `generateContent` REST endpoint using native **Windows WinHTTP** (no external HTTP library).
- `assess_patient()` builds a clinical prompt from the patient's age, gender, symptoms and history, and asks Gemini for a strict JSON response (risk score 0–100, risk level, possible chronic diseases, recommended specialist, symptoms to watch, precautions, immediate action).
- Tries a sequence of **free-tier models** ordered by quota (`gemini-flash-lite-latest` → `gemini-2.5-flash-lite` → `gemini-2.5-flash`), with up to 2 attempts per model.
- Distinguishes two retryable failure types via custom exceptions: `GeminiOverloadedError` (HTTP 503/UNAVAILABLE or no response — worth a short same-model retry with backoff) and `GeminiQuotaExhaustedError` (HTTP 429/RESOURCE_EXHAUSTED — move straight to the next model instead of retrying). Any other error aborts that model and moves on; if every model fails, the whole assessment fails with a descriptive error.

Routes registered:

Method & Path	Purpose
`POST /api/assessments`	Start New Assessment: parses vitals/symptoms/history from the request, calls <font face="

5. db_history.h — Database, Testing & Previous Assessments

Module 3 · Implemented by Anjali

Owns the MySQL connection, schema creation/migration, and all persistence for users and assessments — with an automatic in-memory fallback so the app keeps working if MySQL is unreachable. Also exposes the patient's "show previous assessments" feature and a diagnostic self-test endpoint.

AssessmentMaxHeap — hand-written binary max-heap (DSA)

An array-backed binary max-heap keyed on `Assessment::risk_score`, implemented from scratch (sift-up on insert, sift-down on extract-max) instead of calling `std::sort`. `insert()` and `extract_max()` are each O(log n); `drain_sorted()` repeatedly extracts the max to produce a highest-risk-first ordering in O(n log n) (classic heapsort). It is used only for the in-memory fallback path of `get_all_assessments_sorted()` — when MySQL is connected, the database performs the sort instead via `ORDER BY risk_score DESC`.

DatabaseManager

- `init_database()` connects to MySQL, creates the database and the `users` / `assessments` tables if they don't exist (auto-migration on startup). If the connection fails, it logs a deferral notice and switches the whole manager into an **in-memory fallback mode** instead of crashing.
- `escape()` provides manual SQL-escaping for the MySQL path (`mysql_real_escape_string`) and a hand-written equivalent for the in-memory path.
- `create_user()` / `authenticate_user()` handle sign-up and login against either MySQL or the in-memory `memory_users_` vector, seeded at startup with one demo ASHA worker and one demo patient account.
- `save_assessment()`, `show_previous_assessments(patient_id)` and `get_all_assessments_sorted()` persist and retrieve `Assessment` records, working against either backing store transparently.
- `update_alert()` and `log_contact()` record ASHA worker follow-up actions (alert dispatched / patient contacted) on an assessment.
- All public methods are protected by a `std::mutex` for thread-safety under Crow's multithreaded request handling.

Routes registered:

Method & Path	Purpose
`GET /api/assessments/my?patient_id=`	Patient: Show Previous Assessments — returns that patient's assessment history, most re
`GET /api/db/test`	Database & Testing: reports MySQL connection status and a fixed set of self-test resu

6. dashboard_triage.h — Dashboards & High Risk Triage

Module 4 · Implemented by Nandini

Aggregates statistics for the dashboards and implements the ASHA worker's core triage view: every patient assessment sorted strictly from highest to lowest risk score, so the most urgent cases surface first. Also logs the ASHA worker's "contact patient" outreach action.

Routes registered:

Method & Path	Purpose
`GET /api/worker/assessments`	High Risk Individual prioritization: pulls every assessment via <font face='Courier'>Databas
`POST /api/worker/assessments/<id>/contact`	Contact Patient & outreach logging: records a free-text note (default or custom) again

---

Generated from the current state of backend/src — main.cpp, models.h, auth_notification.h, ai_assessment.h, db_history.h, dashboard_triage.h.